

МИЭМ НИУ ВШЭ

Cross-Team Matcher — лабораторная работа № 2

Документация учебного C++-проекта

Карпов А. К., Лебедев М. О., Кречун А. В.
Версия 4.1.0

Оглавление

Cross-Team Matcher — выполненная лабораторная № 2	2
Структура проекта	2
Проверенный пример	2
Результат документирования	2
Дерево директорий	3
Иерархический список классов	4
Алфавитный указатель классов	5
Список файлов	6
Директории	7
Содержание директории labs/lab2	7
Содержание директории labs	8
Классы	9
Employee	9
RegionalEmployee	12
Файлы	14
labs/lab2/Employee.cpp	14
labs/lab2/Employee.cpp	15
labs/lab2/Employee.h	16
labs/lab2/Employee.h	17
labs/lab2/main.cpp	18
labs/lab2/main.cpp	19
labs/lab2/mainpage.dox	20
Алфавитный указатель	21

Cross-Team Matcher — выполненная лабораторная № 2

Модель сотрудника, недельного бюджета и владельца данных.

Модель сотрудника, недельного бюджета и владельца данных.

Учебный C++17-проект демонстрирует классы предметной области системы подбора команд. [Employee](#) хранит никнейм и недельный бюджет, вычисляет свободные часы. [RegionalEmployee](#) наследует [Employee](#) и хранит регион.

Структура проекта

- [Employee.h](#) — объявления классов, методов и полей;
- [Employee.cpp](#) — реализация и проверки входных значений;
- [main.cpp](#) — запуск и три исполняемые проверки;
- [Employee](#) — базовый класс;
- [RegionalEmployee](#) — производный класс.

Проверенный пример

```
RegionalEmployee employee("synthetic-engineer", 40, 1);
employee.available(24); // 16 свободных часов
employee.available(48); // -8: перегрузка
employee.region();     // 1
```

Результат документирования

Справка сгенерирована из комментариев исходного кода в HTML и RTF. Для файлов, классов и методов приведены краткие и подробные описания, параметры, возвращаемые значения и свойства. Основное приложение использует календарные правила PostgreSQL; C++-пример — отдельная часть лабораторной по инструментам разработки.

Дерево директорий

Алфавитный указатель директорий

lab2	7
Employee.cpp	14
Employee.h	16
main.cpp	18
labs	8
lab2	7
Employee.cpp	14
Employee.h	16
main.cpp	18

Иерархический список классов

Иерархия классов

Иерархия классов.

Employee	9
RegionalEmployee	12

Алфавитный указатель классов

Классы

Классы с их кратким описанием.

<u>Employee</u> (Сотрудник с подтверждённым недельным бюджетом. Поддерживает учебный расчёт свободных часов. В основном приложении расчёт выполняется по каждому рабочему дню внутри PostgreSQL)	9
<u>RegionalEmployee</u> (Сотрудник с неизменяемым домашним регионом. Наследует бюджет и имя <u>Employee</u> , дополняет модель владельцем данных)	12

Список файлов

Файлы

Полный список файлов.

labs/lab2/ Employee.cpp (Реализация учебных Employee и RegionalEmployee)	14
labs/lab2/ Employee.h (Учебная модель сотрудника для демонстрации Doxygen и C++)	16
labs/lab2/ main.cpp (Исполняемая проверка учебного примера для защиты лабораторной)	18

Директории

Содержание директории labs/lab2

Файлы

- файл [Employee.cpp](#)
Реализация учебных [Employee](#) и [RegionalEmployee](#).
- файл [Employee.h](#)
Учебная модель сотрудника для демонстрации Doxygen и C++.
- файл [main.cpp](#)
Исполняемая проверка учебного примера для защиты лабораторной.

Содержание директории labs

Директории

директория [lab2](#)

Классы

Класс Employee

Сотрудник с подтверждённым недельным бюджетом. Поддерживает учебный расчёт свободных часов. В основном приложении расчёт выполняется по каждому рабочему дню внутри PostgreSQL.

```
#include <Employee.h>
```

Граф наследования:Employee:

IMAGE

Открытые члены

- [Employee](#) (std::string nickname, double capacity)
Создать модель.
- virtual [~Employee](#) ()=default
Освободить объект через интерфейс базового класса.
- double [available](#) (double assigned) const
Рассчитать остаток недельного бюджета.
- const std::string & [nickname](#) () const
Рабочее имя.

Защищенные данные

- std::string [nickname](#)
Рабочий никнейм, не закрытое полное имя.
- double [capacity](#)
Подтверждённый недельный бюджет: от 0 до 40 часов.

Подробное описание

Сотрудник с подтверждённым недельным бюджетом. Поддерживает учебный расчёт свободных часов. В основном приложении расчёт выполняется по каждому рабочему дню внутри PostgreSQL.

См. определение в файле [Employee.h](#) строка [13](#)

Конструктор(ы)

Employee::Employee (std::string nickname, double capacity)

Создать модель.

Аргументы

<i>nickname</i>	Рабочее имя без закрытых контактных сведений.
<i>capacity</i>	Подтверждённый бюджет, от 0 до 40 часов.

Исключения

<i>std::invalid_argument</i>	При недопустимом бюджете или имени.
------------------------------	-------------------------------------

См. определение в файле [Employee.cpp](#) строка [10](#)

virtual Employee::~Employee () [virtual], [default]

Освободить объект через интерфейс базового класса.

Методы

double Employee::available (double assigned) const

Рассчитать остаток недельного бюджета.

Метод не изменяет состояние сотрудника. Из подтверждённого бюджета вычитается назначенная нагрузка: `available(24)` при бюджете 40 возвращает 16. При назначении 48 часов результат -8 сохраняет информацию о перегрузке; основной проект проверяет её по дням в SQL.

Аргументы

<i>in</i>	<i>assigned</i>	Уже подтверждённая нагрузка, конечное неотрицательное число.
-----------	-----------------	--

Возвращает

Оставшийся бюджет; отрицательный результат обозначает конфликт.

Исключения

<i>std::invalid_argument</i>	При отрицательной или нечисловой нагрузке.
------------------------------	--

См. определение в файле [Employee.cpp](#) строка [15](#)

const std::string & Employee::nickname () const

Рабочее имя.

Возвращает существующую строку без копирования. Закрытые контактные сведения в учебную модель не включены.

Возвращает

Ссылка на никнейм сотрудника.

См. определение в файле [Employee.cpp](#) строка [19](#)

Данные класса

double Employee::capacity_ [protected]

Подтверждённый недельный бюджет: от 0 до 40 часов.

См. определение в файле [Employee.h](#) строка [41](#)

std::string Employee::nickname_[protected]

Рабочий никнейм, не закрытое полное имя.

См. определение в файле [Employee.h](#) строка [40](#)

Объявления и описания членов классов находятся в файлах:

- labs/lab2/[Employee.h](#)
- labs/lab2/[Employee.cpp](#)

Класс RegionalEmployee

Сотрудник с неизменяемым домашним регионом. Наследует бюджет и имя [Employee](#), дополняет модель владельцем данных.

```
#include <Employee.h>
```

Граф наследования:RegionalEmployee:

IMAGE

Открытые члены

- [RegionalEmployee](#) (std::string nickname, double capacity, int region)
Создать регионального сотрудника.
- int [region](#) () const
Прочитать владельца.

Открытые члены унаследованные от [Employee](#)

- [Employee](#) (std::string nickname, double capacity)
Создать модель.
- virtual [~Employee](#) ()=default
Освободить объект через интерфейс базового класса.
- double [available](#) (double assigned) const
Рассчитать остаток недельного бюджета.
- const std::string & [nickname](#) () const
Рабочее имя.

Закрытые данные

- int [region_](#)
Владелец основной записи сотрудника.

Дополнительные унаследованные члены

Защищенные данные унаследованные от [Employee](#)

- std::string [nickname_](#)
Рабочий никнейм, не закрытое полное имя.
- double [capacity_](#)
Подтверждённый недельный бюджет: от 0 до 40 часов.

Подробное описание

Сотрудник с неизменяемым домашним регионом. Наследует бюджет и имя [Employee](#), дополняет модель владельцем данных.

См. определение в файле [Employee.h](#) строка [47](#)

Конструктор(ы)

RegionalEmployee::RegionalEmployee (std::string nickname, double capacity, int region)

Создать регионального сотрудника.

Аргументы

<i>nickname</i>	Рабочее имя.
<i>capacity</i>	Недельный бюджет.
<i>region</i>	Домашний регион: 1 или 2.

См. определение в файле [Employee.cpp](#) строка [20](#)

Методы

int RegionalEmployee::region () const

Прочитать владельца.

Домашний регион задаётся конструктором и затем не меняется.

Возвращает

Номер домашнего региона.

См. определение в файле [Employee.cpp](#) строка [24](#)

Данные класса

int RegionalEmployee::region_ [private]

Владелец основной записи сотрудника.

См. определение в файле [Employee.h](#) строка [61](#)

Объявления и описания членов классов находятся в файлах:

- labs/lab2/[Employee.h](#)
- labs/lab2/[Employee.cpp](#)

Файлы

Файл labs/lab2/Employee.cpp

Реализация учебных [Employee](#) и [RegionalEmployee](#).

```
#include "Employee.h"  
#include <stdexcept>  
#include <utility>  
#include <cmath>
```

Подробное описание

Реализация учебных [Employee](#) и [RegionalEmployee](#).

Конструкторы проверяют имя, бюджет и номер региона. Метод available вычисляет остаток без изменения объекта. Объявления находятся в [Employee.h](#).

См. определение в файле [Employee.cpp](#)

Employee.cpp

[См. документацию.](#)

```
00001
00006 #include "Employee.h"
00007 #include <stdexcept>
00008 #include <utility>
00009 #include <cmath>
00010 Employee::Employee(std::string nickname, double capacity)
00011     : nickname_(std::move(nickname)), capacity_(capacity) {
00012     if (nickname_.empty() || !std::isfinite(capacity) || capacity < 0 || capacity
00013         > 40)
00014         throw std::invalid_argument("invalid employee capacity");
00015 double Employee::available(double assigned) const {
00016     if (!std::isfinite(assigned) || assigned < 0) throw
00017     std::invalid_argument("negative load");
00018     return capacity_ - assigned;
00019 }
00019 const std::string& Employee::nickname() const { return nickname_; }
00020 RegionalEmployee::RegionalEmployee(std::string nickname, double capacity, int
00021     region)
00022     : Employee(std::move(nickname), capacity), region_(region) {
00023     if (region_ != 1 && region_ != 2) throw std::invalid_argument("invalid region");
00024 }
00024 int RegionalEmployee::region() const { return region_; }
```

Файл labs/lab2/Employee.h

Учебная модель сотрудника для демонстрации Doxygen и C++.
`#include <string>`

Классы

class [Employee](#) *Сотрудник с подтверждённым недельным бюджетом. Поддерживает учебный расчёт свободных часов. В основном приложении расчёт выполняется по каждому рабочему дню внутри PostgreSQL.*

class [RegionalEmployee](#) *Сотрудник с неизменяемым домашним регионом. Наследует бюджет и имя [Employee](#), дополняет модель владельцем данных.*

Подробное описание

Учебная модель сотрудника для демонстрации Doxygen и C++.

Объявляет базовый класс [Employee](#) и производный [RegionalEmployee](#). Разделение интерфейса и реализации показано парой [Employee.h](#) / [Employee.cpp](#).

См. определение в файле [Employee.h](#)

Employee.h

[См. документацию.](#)

```
00001
00006 #pragma once
00007 #include <string>
00008
00013 class Employee {
00014 public:
00020     Employee(std::string nickname, double capacity);
00022     virtual ~Employee() = default;
00032     double available(double assigned) const;
00038     const std::string& nickname() const;
00039 protected:
00040     std::string nickname;
00041     double capacity;
00042 };
00043
00047 class RegionalEmployee : public Employee {
00048 public:
00054     RegionalEmployee(std::string nickname, double capacity, int region);
00059     int region() const;
00060 private:
00061     int region;
00062 };
```

Файл labs/lab2/main.cpp

Исполняемая проверка учебного примера для защиты лабораторной.

```
#include "Employee.h"  
#include <cassert>  
#include <iostream>
```

Функции

- int [main](#) ()
-

Подробное описание

Исполняемая проверка учебного примера для защиты лабораторной.

См. определение в файле [main.cpp](#)

Функции

int main ()

См. определение в файле [main.cpp](#) строка [7](#)

main.cpp

[См. документацию.](#)

```
00001
00004 #include "Employee.h"
00005 #include <cassert>
00006 #include <iostream>
00007 int main() {
00008     RegionalEmployee employee("synthetic-engineer", 40, 1);
00009     assert(employee.available(24) == 16);
00010     assert(employee.region() == 1);
00011     assert(employee.available(48) == -8);
00012     std::cout << "Lab 2: inheritance and capacity checks passed\n";
00013 }
```

Файл labs/lab2/mainpage.dox

Алфавитный указатель

INDEX